

FasTrack Shop — Technical Overview

Quick-commerce grocery delivery platform for Riyadh, Saudi Arabia

Live at shop.fastrack.cloud | Source: github.com/shameerck007/Fastrack-shop | Document generated 2026-09-25

This document is written for a developer who has never touched this codebase before. It covers what the app does, how it's built, where it runs, and what to know before changing anything. Read this fully before making changes to hosting, auth, or the database schema.

1. What this application is

FasTrack Shop is a Phase 1 MVP quick-commerce grocery delivery app with four user roles:

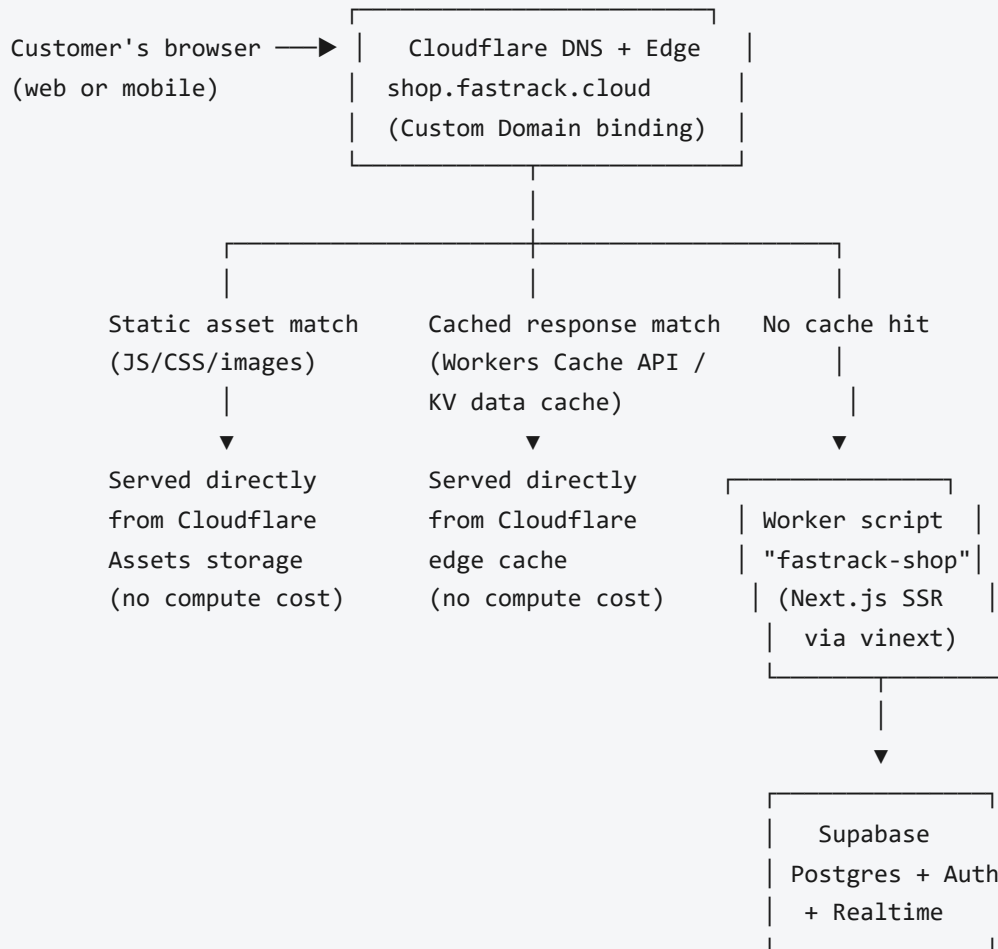
- **Customer** — browse categories, search products, cart, checkout, track orders, delivery OTP.
- **Admin** — catalog/inventory management, order oversight, merchant approval, BI-style dashboard.
- **Merchant** — sell products under the marketplace, manage own product listings.
- **Rider** — view assigned deliveries, update status, OTP-verified drop-off, live location sharing.

The original business requirements live in `BRD_shop.pdf` in the repo root.

2. Tech stack

Layer	Technology
Framework	Next.js 16 (App Router, React Server Components), React 19, TypeScript
Styling	Tailwind CSS v4
Database / Auth	Supabase (Postgres, Auth, Row Level Security, Realtime)
Hosting / Compute	Cloudflare Workers, via the <code>vinext</code> adapter (beta)
Maps	Leaflet + OpenStreetMap tiles
PDF invoices	<code>@react-pdf/renderer</code> + <code>qrcode</code> (ZATCA-style QR for Saudi e-invoicing)
Source control	GitHub — github.com/shameerck007/Fastrack-shop (public)

3. How a request actually flows (architecture)



Key point: most page requests never actually run the Worker's JavaScript — they're served as static assets or from the edge cache. Only requests that need fresh, personalized, or write data (login, cart mutation, checkout, admin edits) invoke the Worker and talk to Supabase.

4. Hosting & deployment details

Item	Value
Cloudflare account	shameerck007@gmail.com
Worker name	fastrack-shop
Custom domain	shop.fastrack.cloud — bound as a Cloudflare Workers Custom Domain (not a CNAME to another host)

Plan	Workers Free — 10 ms CPU time per request limit (see §7)
KV namespace	<code>fastrack-shop-vinext-kv-cache</code> — used for the data cache layer
Static assets	Served via the Workers Assets binding, built into <code>dist/client</code>

Deploying a change

```
git pull
npm install           # only if dependencies changed
npm run build:vinext  # builds the Cloudflare/Workers bundle
npm run deploy:vinext # uploads + activates the new Worker version
```

There is **no CI/CD pipeline**. Deploys are manual, run from a developer's machine with Wrangler already authenticated to the Cloudflare account (`npx wrangler whoami` to check). Pushing to GitHub does **not** update the live site by itself.

Local development

```
npm install
npm run dev          # plain Next.js dev server – fastest for UI work, no Workers runtime
npm run dev:vinext   # vinext dev server – closer to production behaviour
npm run start:vinext # runs the actual built Worker locally via `wrangler dev`
                    # – use this to test anything Workers-specific
                    # (KV, edge cache, cold-start behaviour) before deploying
```

5. Data layer (Supabase)

- Schema + Row Level Security policies: `supabase/migrations/0001_init.sql`
- Sample data: `supabase/seed.sql`
- Roles are stored on the `profiles` table (`customer` , `admin` , `merchant` , `rider`), set by a `handle_new_user` trigger on signup, then promoted manually via SQL — see README for the exact commands.
- Auth cookies are handled by `@supabase/ssr` . Server-side code reads the session via `src/lib/supabase/server.ts` ; client-side/browser code uses `src/lib/supabase/client.ts` .

Auth URL configuration lives only in the Supabase dashboard (Authentication → URL Configuration), not in code. If the domain ever changes, that setting must be updated manually — Site URL and Redirect URLs both need `https://shop.fastrack.cloud` .

6. Project structure

```
src/
  app/
    (storefront)      Routes (Next.js App Router)
                      /, /categories/[slug], /products/[id], /search,
                      /cart, /checkout, /orders, /orders/[id], /addresses,
                      /account, /login, /register
    admin/            /admin, /admin/products, /admin/categories,
                      /admin/orders, /admin/merchants
    merchant/         /merchant, /merchant/products, /sell (apply to sell)
    rider/            /rider, /rider/orders/[id]
    api/              Route handlers: invoice PDF, search suggestions,
                      buy-again feed
  components/         UI components (admin/, rider/, shared)
  lib/
    supabase/         Browser + server Supabase clients
    actions/          Server Actions – the only place that performs writes
                      (cart, checkout, admin CRUD, rider updates, etc.)
    catalog.ts, cart.ts,
    orders.ts, reviews.ts,
    inventory.ts, ... Data-fetching helper functions (reads)
    types/database.ts Hand-written TypeScript types matching the SQL schema
  supabase/
    migrations/0001_init.sql Full schema + RLS
    seed.sql             Sample catalog data
  vite.config.ts        vinext / Cloudflare Workers build configuration
  wrangler.jsonc        Cloudflare Worker + bindings configuration
```

7. Known constraints — read before touching performance-sensitive code

Workers Free plan: 10 ms CPU budget per request

Cloudflare Workers Free gives each request 10 milliseconds of *active* CPU time (network/database wait time doesn't count, only JS execution). A cold-started isolate has to parse and evaluate the app's JS before serving anything, which eats into that budget. This previously caused intermittent **"Error 1102 — Worker exceeded resource limits"** failures, especially on mobile after the app had been idle (a fresh isolate).

Mitigations already in place (as of this document):

- Per-user data (cart count, login state, "Buy Again") was moved out of every page's server render into small client-side components (`UserHeaderActions.tsx` , `BuyAgainSection.tsx`) so most pages no longer call Supabase auth on every server request.

- Cloudflare's Workers Cache API and a KV data cache are enabled (`vite.config.ts`), so repeat requests to the same page can be served from the edge without invoking the Worker at all.
- Build-time route prerendering is enabled where the adapter's static analysis allows it.

If CPU-limit errors reappear, do not add more server-side Supabase calls to components rendered on every page (headers, layouts, nav). Push personalization into client components that fetch after the initial render, and verify locally with `npm run start:vinext` (real Workers runtime, not plain `next dev`) before deploying. Live production behaviour can be watched with `npx wrangler tail fastrack-shop --format pretty` . The permanent fix, if this keeps being a problem, is upgrading to Workers Paid (\$5/mo, 30s+ CPU budget) — this was explicitly deferred by the project owner, not ruled out permanently.

vinext is a beta adapter

`vinext` (Cloudflare's Next.js-on-Workers adapter) is pre-1.0 at the time of writing. Its static-analysis-based route classification (which routes get prerendered vs. run as SSR) is imperfect — don't assume a route is static just because it looks static; check the build output (`npm run build:vinext` prints a route classification table).

8. Environment variables / secrets

Variable	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase project URL (public)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase anonymous/public API key (safe to expose client-side; access is enforced by RLS)

Set in `.env.local` for local dev (copy from `.env.example`). No server-only secrets are currently required — all Supabase access relies on Row Level Security rather than a service-role key.

9. What's not built yet

Per the BRD roadmap (§§26–30, 33–34): multi-vendor marketplace polish, promotions/coupons UI, loyalty/wallet, AI search and shopping assistant, delivery route optimization, subscriptions. The database schema already has room for promotions, reviews, and payments/refunds tables to build on — check `supabase/migrations/0001_init.sql` before adding new tables for these features, they may already partially exist.

10. Quick reference — common tasks

Task	How
Run app locally	<code>npm run dev</code>
Test real Workers behaviour locally	<code>npm run start:vinext</code>
Deploy to production	<code>npm run build:vinext && npm run deploy:vinext</code>
Watch live production logs	<code>npx wrangler tail fastrack-shop --format pretty</code>
Promote a user to admin/rider	SQL in Supabase dashboard — see README.md
Check Cloudflare auth	<code>npx wrangler whoami</code>
Change Supabase Auth redirect URLs	Supabase dashboard → Authentication → URL Configuration (not in code)